

Week10 - CH06.

Dimensionality Reduction

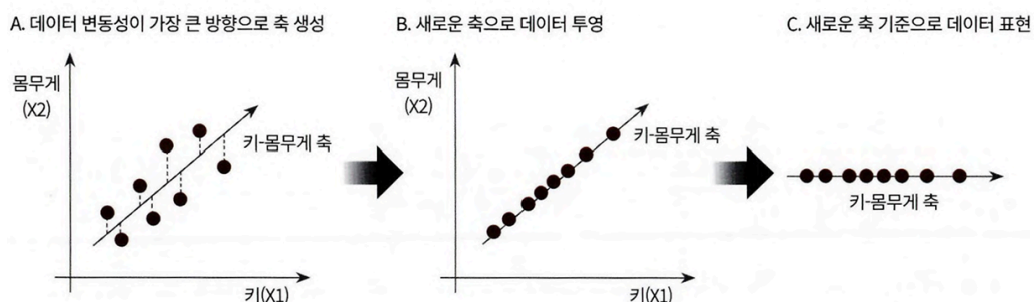
01 차원 축소 개요

- 매우 많은 feature로 구성된 다차원 데이터 세트의 차원을 축소
- 새로운 차원의 데이터 세트를 생성
- 차원 증가할수록 데이터 포인트 간의 거리가 기하급수적으로 멀어짐
 - 희소(sparse)한 구조를 갖게 됨
- 상대적으로 적은 차원에서 학습된 모델보다 예측 신뢰도 낮음
- feature 많은 경우
 - 개별 feature 간 상관관계 높을 가능성 큼
- 선형 회귀 같은 선형 모델
 - 입력 변수 간 상관관계 높은 경우
 - 다중 공선성 문제로 모델의 예측 성능이 저하됨
- 다차원 feature를 차원 축소
 - feature 수 줄이면 더 직관적으로 데이터 해석 가능
- 차원 축소
 - feature selection
 - 특정 feature에 종속성이 강한 불필요한 feature는 아예 제거
 - 데이터 특징 잘 나타내는 주요 feature만 선택
 - feature extraction
 - 기존 feature를 저차원의 중요 feature로 압축하여 추출
 - 새롭게 추출된 중요 feature는 기존의 feature가 압축된 것이므로 완전히 다른 값이 됨
- 단순 압축 아님
- feature를 함축적으로 더 잘 설명할 수 있는 또 다른 공간으로 매핑해 추출하는 것
- 기존 feature가 인지하기 어려웠던 잠재적 요소를 추출할 수 있음

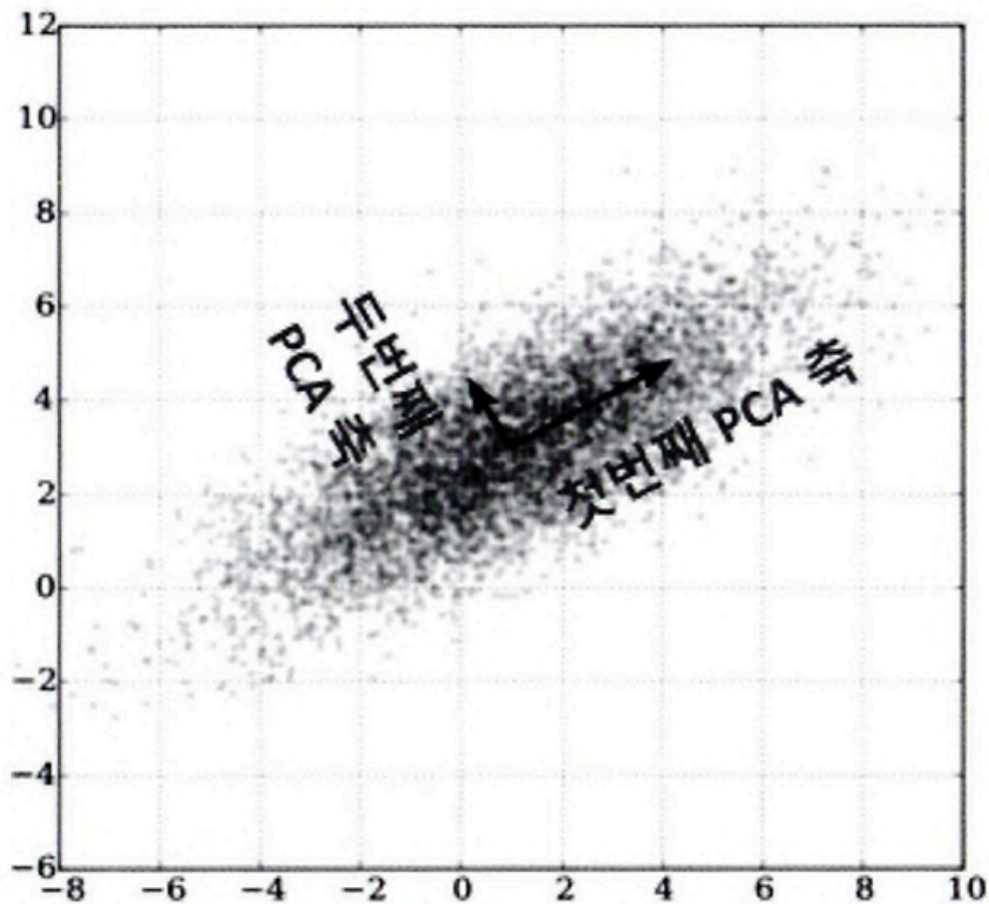
- 잠재적 요소 찾는 대표적 차원 축소 알고리즘
 - PCA
 - SVD
 - NMF
 - 수많은 픽셀로 이뤄진 이미지 데이터에서 잠재된 특성을 feature로 도출해 이미지 변환/압축 가능
 - 원본 이미지보다 훨씬 적은 차원
 - 이미지 분류 등의 분류 수행 시 과적합 영향력 작아짐
 - 텍스트 문서의 숨겨진 의미 추출 가능
 - 문서 내 단어들의 구성에 숨겨진 Semantic 의미 or topic을 잠재적 요소로 간주

02 PCA(Principal Component Analysis)

- 가장 대표적인 차원 축소 기법
- 여러 변수 간에 존재하는 상관관계 이용해 대표하는 주 성분(Principal component)을 추출해 차원 축소
- 기존 데이터의 정보 유실이 최소화되는 것이 당연
 - PCA는 가장 높은 분산을 가지는 데이터의 축을 찾아 이 축으로 차원 축소
 - 이것이 PCA의 주성분
 - → 분산이 데이터의 특성을 가장 잘 나타내는 것으로 간주



- 2개의 feature를 한 개의 주성분을 가진 데이터 세트로 차원 축소 가능
 - 데이터 변동성이 가장 큰 방향으로 축을 생성
 - 새롭게 생성된 축으로 데이터 투영



- 먼저 가장 큰 데이터 변동성을 기반으로 첫 번째 벡터 축 생성
- 두 번째 축은 이 축에 직각이 되는 벡터를 설정하는 방식으로 축 생성
- 이 벡터에 원본 데이터 투영하면 벡터 축의 개수만큼의 차원으로 원본 데이터가 차원 축소됨
- PCA의 선형대수 관점
 - 입력 데이터의 공분산 행렬(Covariance Matrix)을 고윳값 분해
 - → 구한 고유벡터에 입력 데이터를 선형 변환하는 것
 - 이 고유 벡터가 PCA의 주성분 벡터로서 입력 데이터의 분산이 큰 방향을 나타냄

- 고윳값
 - 고유 벡터의 크기 & 입력 데이터의 분산
- 일반적인 선형 변환
 - 특정 벡터에 행렬 A를 곱해 새로운 벡터로 변환하는 것
 - 특정 벡터를 하나의 공간에서 다른 공간으로 투영하는 개념
 - 이 경우 이 행렬을 바로 공간으로 가정
- 보통 분산은 한 개의 특정한 변수의 데이터의 변동 의미
- 공분산은 두 변수 간의 변동을 의미
- 공분산 행렬은 여러 변수와 관련된 공분산을 포함하는 정방형 행렬
- 고유 벡터는 행렬A를 곱하더라도 방향이 변하지 않고 그 크기만 변하는 벡터
 - $Ax = ax$
 - A는 행렬, x는 고유 벡터, a는 스칼라값
 - 고유벡터는 여러 개 존재
 - 정방 행렬은 최대 그 차원 수만큼의 고유 벡터를 가질 수 있음
 - 고유 벡터는 행렬이 작용하는 힘의 방향과 관계 있음
 - → 행렬 분해하는 데에 사용
- 공분산 행렬은 정방행렬이며 대칭행렬
- 대칭 행렬은 항상 고유 벡터를 직교행렬로, 고윳값을 정방 행렬로 대각화할 수 있음
- 입력 데이터의 공분산 행렬을 C라 하면 공분산 행렬의 특성으로 분해할 수 있음

$$C = P \Sigma P^T$$

$$C = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^t \\ \cdots \\ e_n^t \end{bmatrix}$$

- P는 nxn 직교 행렬
- 입력 데이터의 공분산 행렬이 고유벡터와 고윳값으로 분해될 수 있음
- 이렇게 분해된 고유 벡터를 이용해 입력 데이터를 선형 변환하는 방식이 PCA
- PCA의 절차

1. 입력 데이터 세트의 공분산 행렬을 생성합니다.
2. 공분산 행렬의 고유벡터와 고유값을 계산합니다.
3. 고유값이 가장 큰 순으로 K개(PCA 변환 차수만큼)만큼 고유벡터를 추출합니다.
4. 고유값이 가장 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환합니다.

```

제안된 코드에 라이선스가 적용될 수 있습니다.
from sklearn.datasets import load_iris
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

iris = load_iris()

columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
irisDF = pd.DataFrame(iris.data, columns=columns)
irisDF['target'] = iris.target
irisDF.head(3)

```

	sepal_length	sepal_width	petal_length	petal_width	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0

- 붓꽃 데이터 세트 로딩 후 DataFrmae으로 변환

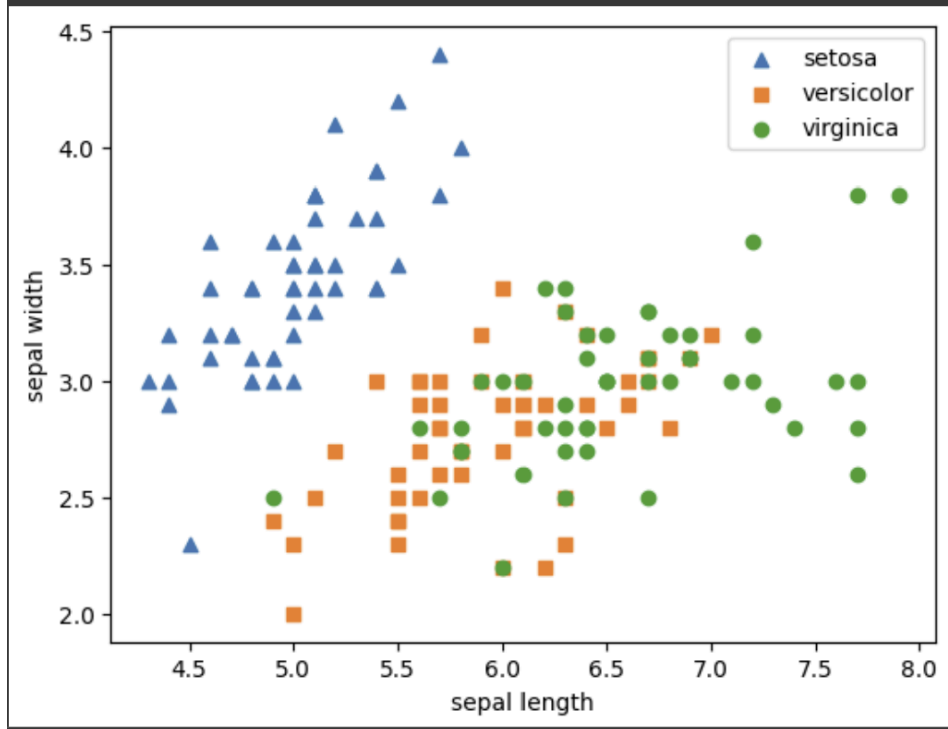
```

markers=['^', 's', 'o']

for i, marker in enumerate(markers):
    x_axis_data = irisDF[irisDF['target'] == i]['sepal_length']
    y_axis_data = irisDF[irisDF['target'] == i]['sepal_width']
    plt.scatter(x_axis_data, y_axis_data, marker=marker, label=iris.target_names[i])

plt.legend()
plt.xlabel('sepal length')
plt.ylabel('sepal width')
plt.show()

```



- 각 품종에 따라 분포를 2차원으로 시각화

```

from sklearn.preprocessing import StandardScaler

iris_scaled = StandardScaler().fit_transform(irisDF.iloc[:, :-1])

from sklearn.decomposition import PCA

pca = PCA(n_components=2)

pca.fit(iris_scaled)
iris_pca = pca.transform(iris_scaled)
print(iris_pca.shape)

(150, 2)

제안된 코드에 라이선스가 적용될 수 있습니다. | cieabora/cieabora.github.io
pca_columns=['pca_component_1', 'pca_component_2']
irisDF_pca = pd.DataFrame(iris_pca, columns=pca_columns)
irisDF_pca['target'] = iris.target
irisDF_pca.head(3)

```

	pca_component_1	pca_component_2	target
0	-2.264703	0.480027	0
1	-2.080961	-0.674134	0
2	-2.364229	-0.341908	0

- 바로 PCA 적용하기 전 개별 속성을 함께 스케일링
- PCA는 여러 속성의 값을 연산
 - 속성의 스케일에 영향 받음
 - 여러 속성을 PCA로 압축하기 전 각 속성값을 동일한 스케일로 변환하는 것이 필요
- 스케일링된 데이터 세트에 PCA 적용

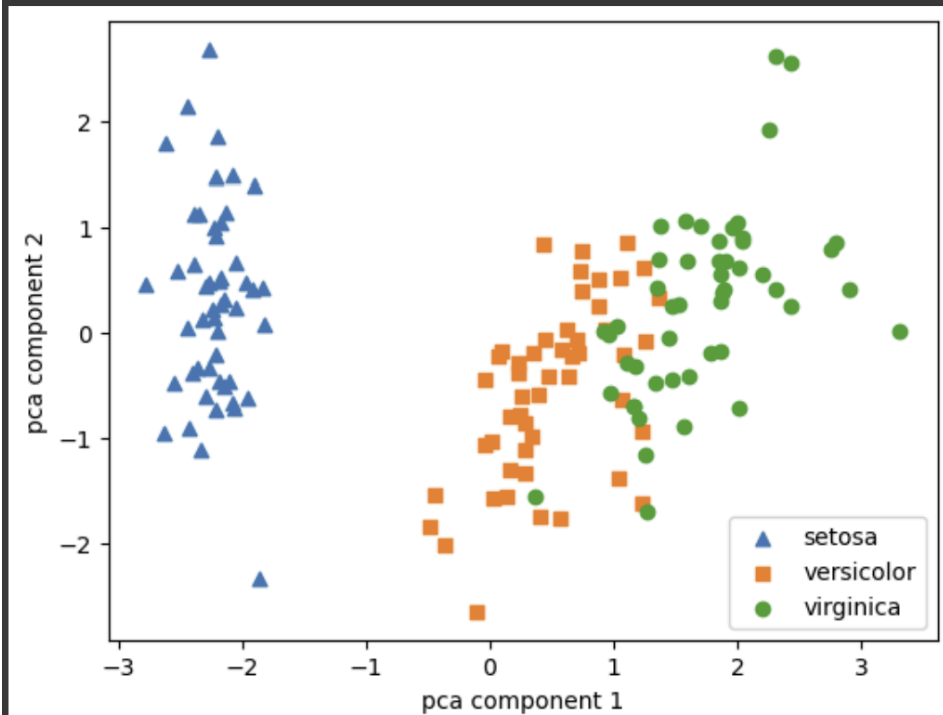
```

markers=['^', 's', 'o']

for i, marker in enumerate(markers):
    x_axis_data = irisDF_pca[irisDF_pca['target'] == i]['pca_component_1']
    y_axis_data = irisDF_pca[irisDF_pca['target'] == i]['pca_component_2']
    plt.scatter(x_axis_data, y_axis_data, marker=marker, label=iris.target_names[i])

plt.legend()
plt.xlabel('pca component 1')
plt.ylabel('pca component 2')
plt.show()

```



- 2개의 속성으로 PCA 변환된 데이터 세트를 2차원상에서 시각화
- pca_component_1 속성을 X축
- pca_component_2 속성을 Y축으로
- 대체로 구분이 잘 되는 이유는 PCA의 첫 번째 새로운 축인 pca_component_1이 원본 데이터의 변동성 잘 반영함


```

print(pca.explained_variance_ratio_)

[0.72962445 0.22850762]

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

rcf = RandomForestClassifier(random_state=156)
scores = cross_val_score(rcf, iris.data, iris.target, scoring='accuracy', cv=3)
print('원본 데이터 교차 검증 개별 정확도', scores)
print("원본 데이터 평균 정확도:", np.mean(scores))

원본 데이터 교차 검증 개별 정확도" [0.98 0.94 0.96]
원본 데이터 평균 정확도: 0.96

제안된 코드에 라이선스가 적용될 수 있습니다. | cieabora/cieabora.github.io
pca_X = irisDF_pca[['pca_component_1', 'pca_component_2']]
scores_pca = cross_val_score(rcf, pca_X, iris.target, scoring='accuracy', cv=3)
print('PCA 변환 데이터 교차 검증 개별 정확도', scores_pca)
print('PCA 변환 데이터 평균 정확도:', np.mean(scores_pca))

PCA 변환 데이터 교차 검증 개별 정확도" [0.88 0.88 0.88]
PCA 변환 데이터 평균 정확도: 0.88

```

- 원본 데이터의 변동성 얼마나 반영하는지 확인
- explained_variance_ratio_ 속성은 전체 변동성 중 개별 PCA Component별로 차지하는 변동성 비율 제공
- 원본 붓꽃 데이터에 랜덤 포레스트 적용
- 기존 4차원 데이터를 2차원으로 PCA 변환한 데이터 세트에 랜덤 포레스트 적용
- 원본 데이터 세트 대비 예측 정확도는 PCA 변환 차원 개수에 따라 예측 성능 떨어질 수 밖에 없음
- 4개의 속성이 2개로 감소
 - 예측 성능의 정확도 8%하락
 - 속성 개수가 50% 감소한 것에 비해 원본 데이터 특성 상당히 유지됨

```
import pandas as pd
```

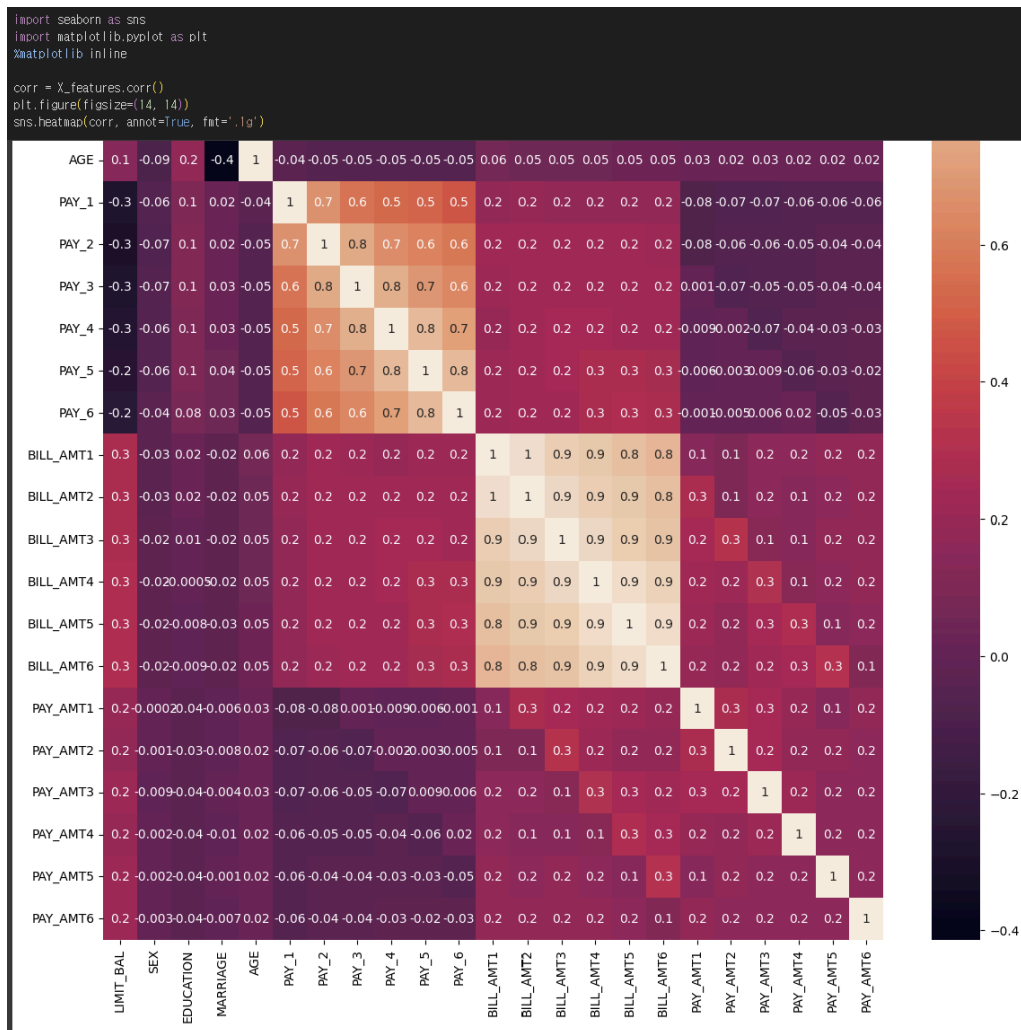
```
df = pd.read_excel('pca_credit_card.xls', header=1, sheet_name='Data').iloc[0: , 1: ]  
print(df.shape)  
df.head
```

```
(30000, 24)
```

```
on position. It is useful for quickly testing if your object  
has the right type of data in it.
```

```
df.rename(columns={"PAY_0":'PAY_1', 'default payment next month':'default'}, inplace=True)  
y_target = df['default']  
X_features = df.drop('default', axis=1)
```

- 30,000개의 레코드와 24개의 속성
- 'default payment next month' 속성이 Target 값으로 '다음달 연체 여부' 의미
 - '연체' → 1, '정상납부' → 0
- PAY_0 다음에 PAY_2 칼럼이므로 PAY_0을 PAY_1로 칼럼명 변환
- default payment next month → default로 칼럼명 변경
- Target 속성인 'default' 칼럼을 y_target 변수로 별도 지정
- feature 데이터는 default 칼럼 제외 별도의 DataFrame으로 만듦



- DataFrame의 corr())를 이용하여 각 속성 간의 상관도 구함
- → Seaborn의 heatmap으로 시각화
- BILL_AMT1 ~ BILL_AMT6 6개 속성끼리 상관도 매우 높음
- 높은 상관도 가진 속성들은 소수의 PCA만으로도 자연스럽게 속성들의 변동성 수용 가능

```

from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

cols_bill = ['BILL_AMT'+str(i) for i in range(1, 7)]
print('대상 속성명:', cols_bill)

scaler = StandardScaler()
df_cols_scaled = scaler.fit_transform(X_features[cols_bill])
pca = PCA(n_components=2)
pca.fit(df_cols_scaled)
print('PCA Component별 변동성:', pca.explained_variance_ratio_)

대상 속성명: ['BILL_AMT1', 'BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6']
PCA Component별 변동성: [0.90555253 0.0509867 ]

import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

rfc = RandomForestClassifier(n_estimators=300, random_state=156)
scores = cross_val_score(rfc, X_features, y_target, scoring='accuracy', cv=3)

print('CV=3인 경우의 개별 Fold 세트별 정확도:', scores)
print('평균 정확도:{0:.4f}'.format(np.mean(scores)))

CV=3인 경우의 개별 Fold 세트별 정확도: [0.8083 0.8196 0.8232]
평균 정확도:0.8170

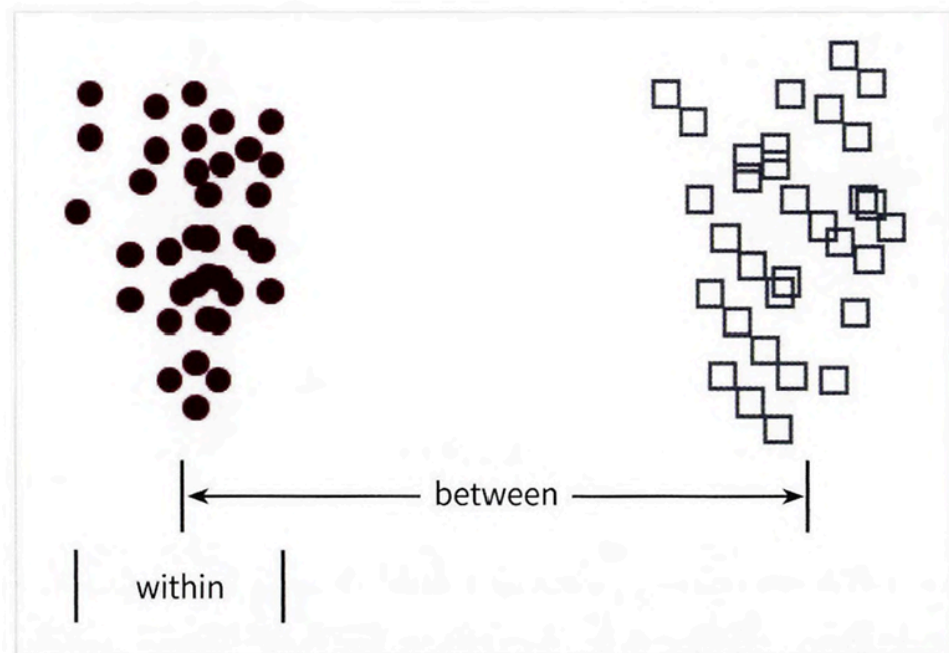
```

- BILL_AMT1 ~ BILL_AMT6까지 6개 속성으로 2개의 component로 변환
- 개별 component의 변동성을 explained_variance_ratio 속성으로 알아봄
- 2개의 PCA component만으로도 6개 속성의 변동성을 약 95% 이상 설명할 수 있음
- 첫 번째 PCA 축으로 90%의 변동성을 수용할 정도로 6개 속성의 상관도 매우 높음
- 원본 데이터 세트와 6개의 component로 PCA 변환한 데이터 세트의 분류 예측 결과를 상화 비교
- 원본 데이터 세트에 랜덤 포레스트 이용해 타깃 값이 디폴트 값으로 3개의 교차 검증 세트로 분류 예측
- 6개의 component로 PCA 변환한 데이터 세트에 대해 동일하게 분류 예측 적용
- 전체 23개 속성의 약 1/4 수준인 6개의 PCA component만으로도 원본 데이터를 기반으로 한 분류 예측 결과보다 약 1-2% 정도의 예측 성능 저하만 발생
- PCA의 뛰어난 압축 능력

03 LDA(Linear Discriminant Analysis)

LDA 개요

- 선형 판별 분석법
- PCA와 매우 유사
 - 입력 데이터 세트를 저차원 공간에 투영해 차원을 축소하는 기법
- 중요한 차이는 LDA는 classification에서 사용하기 쉽도록 개별 클래스 분별 가능 기준을 최대한 유지하면서 차원 축소
- PCA는 입력 데이터의 변동성 가장 큰 축 찾음
- LDA는 입력 데이터의 결정 값 클래스를 최대한으로 분리할 수 있는 축 찾음
- LDA는 특정 공간 상 클래스 분리를 최대화하는 축 찾기 위한 방식으로 차원 축소
 - 클래스 간 분산 최대한 크게
 - 클래스 내부 분산 최대한 작게



- LDA 구하는 스텝

1. 클래스 내부와 클래스 간 분산 행렬을 구합니다. 이 두 개의 행렬은 입력 데이터의 결정 값 클래스별로 개별 피처의 평균 벡터(mean vector)를 기반으로 구합니다.
2. 클래스 내부 분산 행렬을 S_W , 클래스 간 분산 행렬을 S_B 라고 하면 다음 식으로 두 행렬을 고유벡터로 분해할 수 있습니다.

$$S_W^T S_B = \begin{bmatrix} e_1 & \cdots & e_n \end{bmatrix} \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^T \\ \cdots \\ e_n^T \end{bmatrix}$$

3. 고유값이 가장 큰 순으로 K개(LDA변환 차수만큼) 추출합니다.
4. 고유값이 가장 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환합니다.

붓꽃 데이터 세트에 LDA 적용하기

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

iris = load_iris()
iris_scaled = StandardScaler().fit_transform(iris.data)

lda = LinearDiscriminantAnalysis(n_components=2)
lda.fit(iris_scaled, iris.target)
iris_lda = lda.transform(iris_scaled)
print(iris_lda.shape)

(150, 2)
```

- 사이킷런은 LDA를 LinearDiscriminant Analysis 클래스로 제공
- 2개의 component로 붓꽃 데이터를 LDA 변환
- PCA와 다르게 LDA에서 유의할 점
 - LDA는 실제로 PCA와 다르게 비지도 학습이 아닌 지도 학습
 - 즉 클래스 결정 값이 변환 시에 필요함
 - lda 객체의 fit() 메서드를 호출할 때 결정값이 입력됐음에 유의하기

```

import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

lda_columns=['lda_component_1', 'lda_component_2']
irisDF_lda = pd.DataFrame(iris_lda, columns=lda_columns)
irisDF_lda['target'] = iris.target

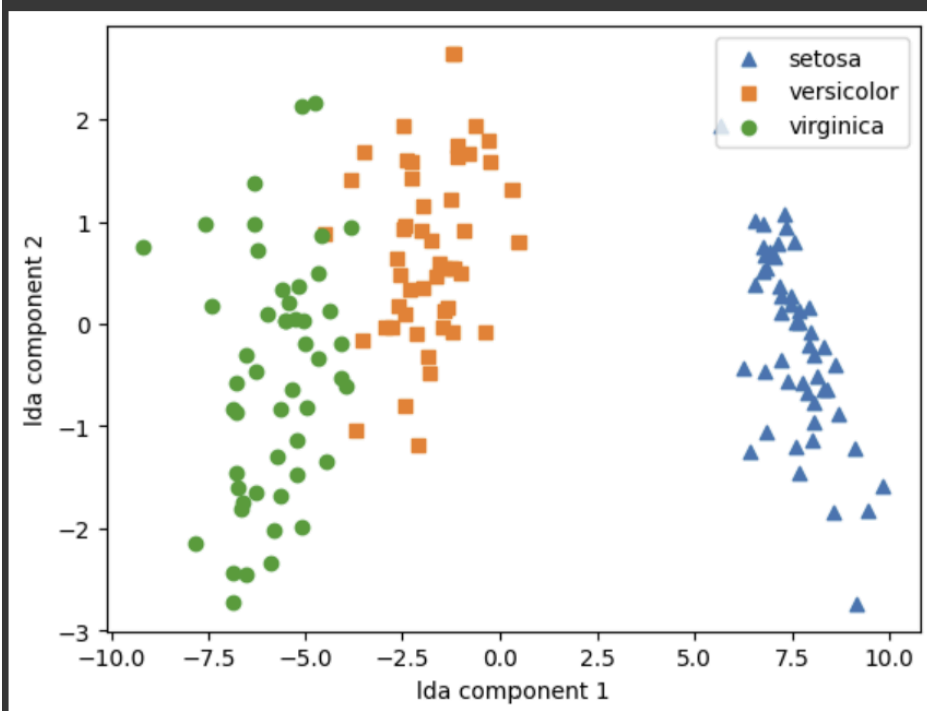
markers = ['^', 's', 'o']

for i, marker in enumerate(markers):
    x_axis_data = irisDF_lda[irisDF_lda['target'] == i]['lda_component_1']
    y_axis_data = irisDF_lda[irisDF_lda['target'] == i]['lda_component_2']

    plt.scatter(x_axis_data, y_axis_data, marker=marker, label=iris.target_names[i])

plt.legend(loc='upper right')
plt.xlabel('lda component 1')
plt.ylabel('lda component 2')
plt.show()

```



- LDA 변환된 입력 데이터 값을 2차원 평면에 품종별로 표현
- PCA로 변환된 데이터와 좌우 대칭 형태로 많이 닮음

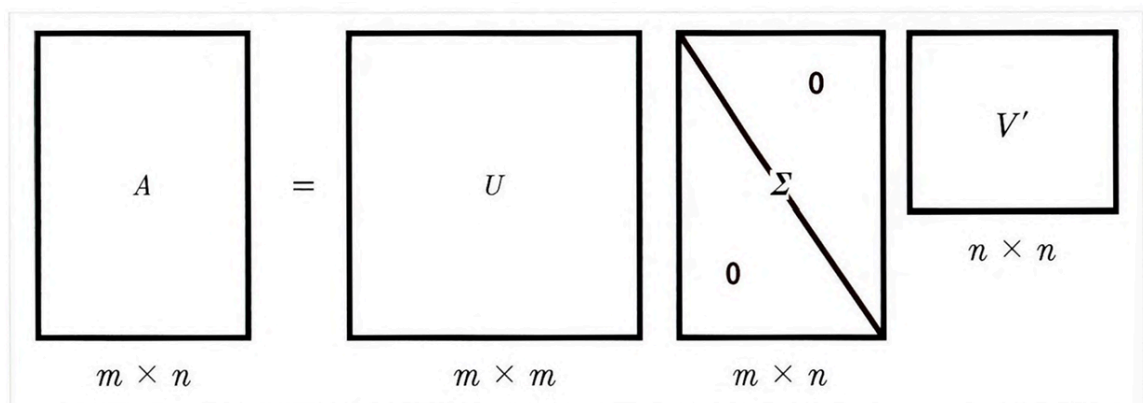
04 SVD(Singular Value Decomposition)

SVD 개요

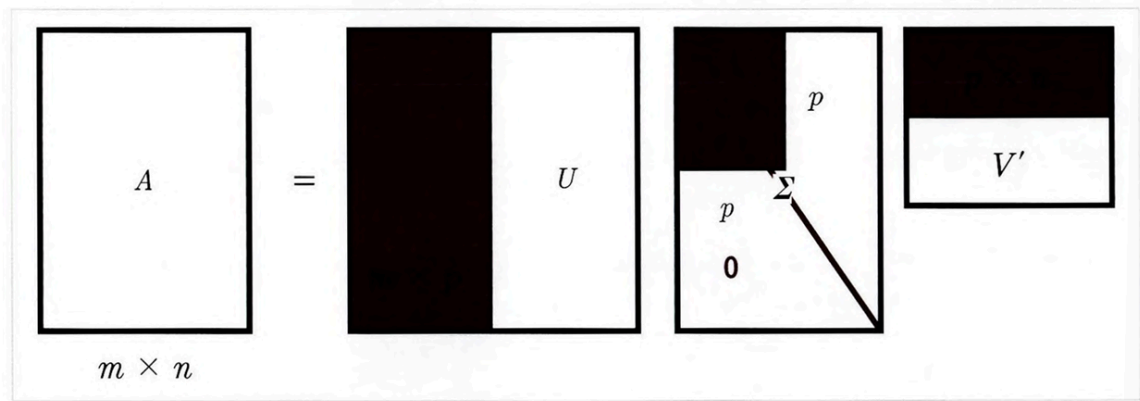
- PCA와 유사한 행렬 분해 기법 이용
- PCA는 정방행렬만 고유 벡터로 분해 가능
- SVD는 정방행렬뿐 아니라 행/열 크기 달라도 적용 가능
- 일반적으로 SVD는 $m \times n$ 크기 행렬 분해하는 것 의미

$$A = U \Sigma V^T$$

- 특이값 분해로 불림
- 행렬 U 와 C 에 속한 벡터는 특이벡터
- 모든 특이 벡터는 서로 직교함
- Σ 는 대각행렬
- Σ 이 위치한 0이 아닌 값이 행렬 A 의 특이값
- SVD는 A 의 차원이 $m \times n$ 일 때 U 차원이 $m \times m$, V^T 의 차원이 $n \times n$ 으로 분해



- 일반적으로 Σ 의 비대각인 부분과 대각원소 중 특이값이 0인 부분도 모두 제거
- 제거된 Σ 에 대응되는 U , V 원소도 함께 제거
 - 차원 줄인 형태로 SVD 적용
- 컴팩트한 형태로 SVD 적용하면 A 의 차원이 $m \times n$ 일 때, U 차원을 $m \times p$, Σ 차원을 $p \times p$, V^T 의 차원을 $p \times n$ 으로 분해



- Truncated SVD는 Σ 의 대각원소 중 상위 몇 개만 추출
 - 여기에 대응하는 U 와 V 의 원소도 함께 제거
 - 더욱 차원 줄인 형태로 분해
- 일반적 SVD는 보통 넘파이나 사이파이 라이브러리 이용해 수행

```

import numpy as np
from numpy.linalg import svd

np.random.seed(121)
a = np.random.randn(4,4)
print(np.round(a,3))

[[-0.212 -0.285 -0.574 -0.44 ]
 [-0.33   1.184  1.615  0.367]
 [-0.014  0.63   1.71  -1.327]
 [ 0.402 -0.191  1.404 -1.969]]

U, Sigma, Vt = svd(a)
print(U.shape, Sigma.shape, Vt.shape)
print('U matrix:###n', np.round(U, 3))
print('Sigma Value:###n', np.round(Sigma, 3))
print('V transpose matrix:###n', np.round(Vt, 3))

(4, 4) (4,) (4, 4)
U matrix:
[[-0.079 -0.318  0.867  0.376]
 [ 0.383  0.787  0.12   0.469]
 [ 0.656  0.022  0.357 -0.664]
 [ 0.645 -0.529 -0.328  0.444]]
Sigma Value:
[3.423 2.023 0.463 0.079]
V transpose matrix:
[[ 0.041  0.224  0.786 -0.574]
 [-0.2    0.562  0.37   0.712]
 [-0.778  0.395 -0.333 -0.357]
 [-0.593 -0.692  0.366  0.189]]

```

- 랜덤 행렬 생성
 - 행렬의 개별 로우끼리 의존성 없애기 위해

- a 행렬에 SVD를 적용해 U, Sigma, Vt를 도출
- SVD 분해는 `numpy.linalg.svd`에 파라미터로 원본 행렬 입력
 - U 행렬, Sigma행렬, Vt행렬 반환
- Sigma 행렬은 행렬의 대각에 위치한 값만 0이 아님
 - 그렇지 않은 경우 모두 0이므로 0이 아닌 값의 경우만 1차원 행렬로 표현

```
Sigma_mat = np.diag(Sigma)
a_ = np.dot(np.dot(U, Sigma_mat), Vt)
print(np.round(a_, 3))
```

```
[[ -0.212  -0.285  -0.574  -0.44 ]
 [ -0.33    1.184   1.615   0.367]
 [ -0.014   0.63    1.71    -1.327]
 [  0.402  -0.191   1.404  -1.969]]
```

```
a[2] = a[0] + a[1]
a[3] = a[0]
print(np.round(a, 3))
```

```
[[ -0.212  -0.285  -0.574  -0.44 ]
 [ -0.33    1.184   1.615   0.367]
 [ -0.542   0.899   1.041  -0.073]
 [ -0.212  -0.285  -0.574  -0.44 ]]
```

```
U, Sigma, Vt = svd(a)
print(U.shape, Sigma.shape, Vt.shape)
print('Sigma Value:\n', np.round(Sigma, 3))
```

```
(4, 4) (4,) (4, 4)
Sigma Value:
[2.663 0.807 0.    0.   ]
```

- 분해된 U, Sigma, Vt를 이용해 다시 원본 행렬로 정확히 복원되는지 확인
- 원본 행렬로의 복원은 U, Sigma, Vt를 내적하면 됨
- Sigma의 겨우 유의점
 - 0이 아닌 값만 1차원으로 추출했으므로 다시 0을 포함한 대칭행렬로 변환한 뒤 내적 수행해야 됨

- $a_{\cdot}$ 는 원본 행렬 a 와 동일하게 복원됨
- 데이터 세트가 로우 간 의존성 있을 경우 Sigma 값 어떻게 변화?
 - 그에 따른 차원 축소 진행?
- a 행렬은 전과 달리 로우 간 관계 매우 높아짐
- 이전과 차원은 같으나 Sigma 값 중 2개가 0으로 변함
- 선형 독립인 로우 벡터의 개수가 2개
 - 행렬의 랭크가 2

```

U_ = U[:, :2]
Sigma_ = np.diag(Sigma[:2])

Vt_ = Vt[:2]
print(U_.shape, Sigma_.shape, Vt_.shape)

a_ = np.dot(np.dot(U_, Sigma_), Vt_)
print(np.round(a_, 3))

(4, 2) (2, 2) (2, 4)
[[-0.212 -0.285 -0.574 -0.44 ]
 [-0.33   1.184  1.615  0.367]
 [-0.542  0.899  1.041 -0.073]
 [-0.212 -0.285 -0.574 -0.44 ]]

```

- 분해된 U , Sigma , Vt 를 이용해 다시 원본 행렬로 복원
- 전체 데이터를 이용하지 않고 Sigma 의 0에 대응되는 U , Sigma , Vt 의 데이터를 제외하고 복원
- Sigma 경우 앞의 2개 요소만 0이 아니므로 U 행렬 중 선행 두 개의 열만 추출, Vt 의 경우 선행 두 개의 행만 추출해 복원

```

import numpy as np
from scipy.sparse.linalg import svds
from scipy.linalg import svd

np.random.seed(121)
matrix = np.random.random((6, 6))
print('원본 행렬:\n', matrix)
U, Sigma, Vt = svd(matrix, full_matrices=False)
print('\n분해 행렬 차원:', U.shape, Sigma.shape, Vt.shape)
print('\nSigma값 행렬:', Sigma)

num_components = 4
U_tr, Sigma_tr, Vt_tr = svds(matrix, k=num_components)
print('\nTruncated SVD 분해 행렬 차원:', U_tr.shape, Sigma_tr.shape, Vt_tr.shape)
print('\nTruncated SVD Sigma값 행렬:', Sigma_tr)
matrix_tr = np.dot(np.dot(U_tr, np.diag(Sigma_tr)), Vt_tr)

print('\nTruncated SVD로 분해 후 복원 행렬:\n', matrix_tr)

```

원본 행렬:

```

[[0.11133083 0.21076757 0.23296249 0.15194456 0.83017814 0.40791941]
 [0.5557906  0.74552394 0.24849976 0.9686594  0.95268418 0.48984885]
 [0.01829731 0.85760612 0.40493829 0.62247394 0.29537149 0.92958852]
 [0.4056155  0.56730065 0.24575605 0.22573721 0.03827786 0.58098021]
 [0.82925331 0.77326256 0.94693849 0.73632338 0.67328275 0.74517176]
 [0.51161442 0.46920965 0.6439515  0.82081228 0.14548493 0.01806415]]

```

분해 행렬 차원: (6, 6) (6,) (6, 6)

Sigma값 행렬: [3.2535007 0.88116505 0.83865238 0.55463089 0.35834824 0.0349925]

Truncated SVD 분해 행렬 차원: (6, 4) (4,) (4, 6)

Truncated SVD Sigma값 행렬: [0.55463089 0.83865238 0.88116505 3.2535007]

Truncated SVD로 분해 후 복원 행렬:

```

[[0.19222941 0.21792946 0.15951023 0.14084013 0.81641405 0.42533093]
 [0.44874275 0.72204422 0.34594106 0.99148577 0.96866325 0.4754868 ]
 [0.12656662 0.88860729 0.30625735 0.59517439 0.28036734 0.93961948]
 [0.23989012 0.51026588 0.39697353 0.27308905 0.05971563 0.57156395]
 [0.83806144 0.78847467 0.93868685 0.72673231 0.6740867  0.73812389]
 [0.59726589 0.47953891 0.56613544 0.80746028 0.13135039 0.03479656]]

```

- Truncated SVD를 이용해 행렬 분해
- Truncated SVD는 특이값 중 상위 일부 데이터만 추출해 분해하는 방식
- 인위적으로 더 작은 차원의 U , Σ , V^*T 로 분해하므로 원본 행렬을 정확히 다시 원복할 수는 없음
- 데이터 정보가 압축되어 분해됨에도 원본 행렬을 상당 수준 근사할 수 있음
- 원래 차원 차수에 가깝게 잘라낼수록 원본 행렬에 더 가깝게 복원 가능
- Truncated SVD는 사이파이에서만 지원됨

- Truncated SVD로 분해된 행렬로 다시 복원할 경우 완벽하게 복원되지 않고 근사적으로 복원됨

사이킷런 TruncatedSVD 클래스를 이용해 변환

- 사이킷런의 TruncatedSVD 클래스는 사이파이의 svds와 같이 Truncated SVD 연산을 수행해 원본 행렬을 분해한 U, Sigma, Vt 행렬을 반환하진 않음
- 사이킷런의 TruncatedSVD 클래스는 PCA 클래스와 유사
 - fit(), transform()을 호출해 원본 데이터를 몇 개의 주요 component로 차원 축소해 변환
- 원본 데이터를 Truncated SVD 방식으로 분해된 $U \cdot \Sigma$ 행렬에 선형 변환해 생성

```

from sklearn.decomposition import TruncatedSVD, PCA
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
%matplotlib inline

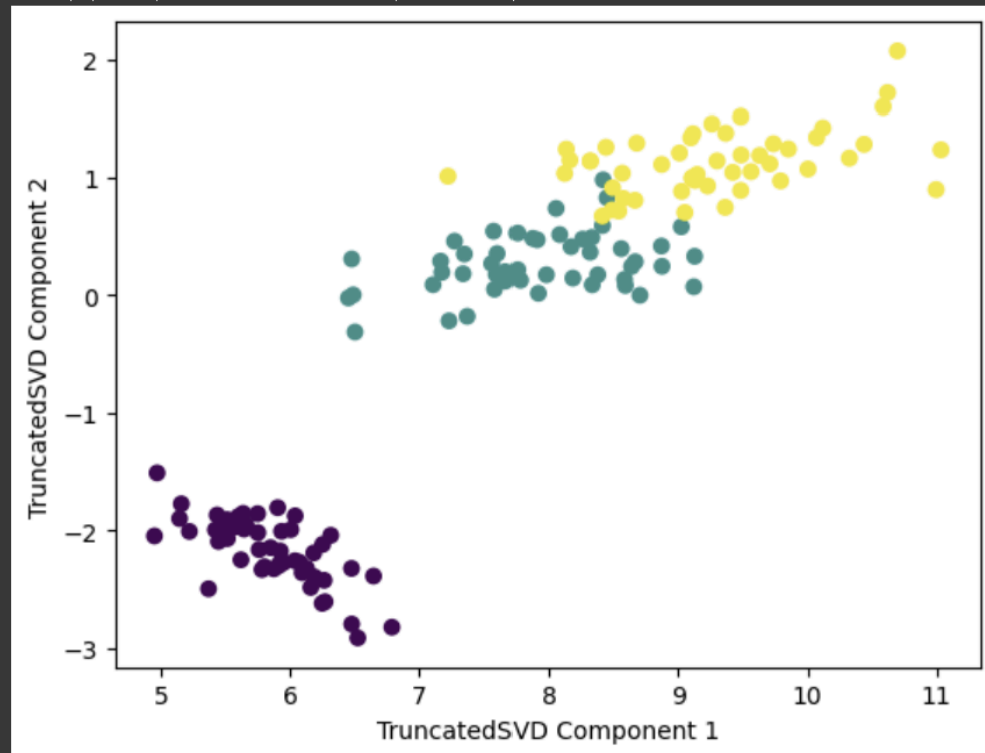
iris = load_iris()
iris_ftsr = iris.data

tsvd = TruncatedSVD(n_components=2)
tsvd.fit(iris_ftsr)
iris_tsvd = tsvd.transform(iris_ftsr)

plt.scatter(x=iris_tsvd[:, 0], y = iris_tsvd[:, 1], c = iris.target)
plt.xlabel('TruncatedSVD Component 1')
plt.ylabel('TruncatedSVD Component 2')

Text(0, 0.5, 'TruncatedSVD Component 2')

```




```

from sklearn.preprocessing import StandardScaler

sclaer = StandardScaler()
iris_scaled = sclaer.fit_transform(iris_ftrs)

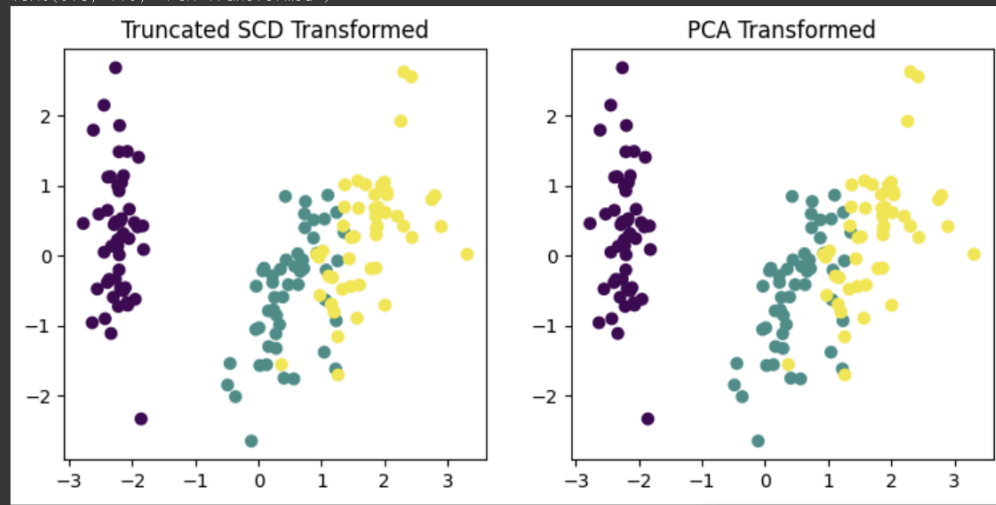
tsvd = TruncatedSVD(n_components=2)
tsvd.fit(iris_scaled)
iris_tsvd = tsvd.fit_transform(iris_scaled)

pca = PCA(n_components=2)
pca.fit(iris_scaled)
iris_pca = pca.fit_transform(iris_scaled)

fig, (ax1, ax2) = plt.subplots(figsize=(9, 4), ncols=2)
ax1.scatter(x=iris_tsvd[:, 0], y=iris_tsvd[:, 1], c=iris.target)
ax2.scatter(x=iris_pca[:, 0], y=iris_pca[:, 1], c=iris.target)
ax1.set_title('Truncated SCD Transformed')
ax2.set_title('PCA Transformed')

```

Text(0.5, 1.0, 'PCA Transformed')



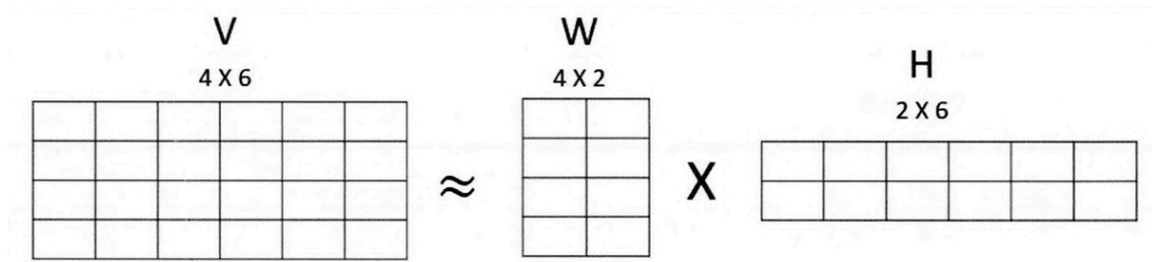
- 두 개의 변환 행렬 값과 원본 속성별 컴포넌트 비율값을 실제로 서로 비교해 보면 거의 같음
- 데이터 세트가 스케일링으로 데이터 중심 동일해지면 사이킷런의 SVD와 PC는 동일한 변환 수행
 - PCA가 SVD 알고리즘으로 구현됐음을 의미
- PCA는 밀집 행렬(Dense Matrix)에 대한 변환만 가능
- SVD는 희소 행렬에 대한 변환 가능

05 NMF(Non-Negative-Matrix Factorization)

NMF 개요

- Truncated SVD처럼 낮은 랭크 통한 행렬 근사 방식의 변형

- NMF는 원본 행렬 내의 모든 원소 값이 모두 양수(0 이상)



- 4×6 원본 행렬 V는 행렬 W와 행렬 H로 근사해 분해될 수 있음
- 행렬 분해는 일반적으로 SVD와 같은 행렬 분해 기법을 통칭하는 것
- 행렬 분해를 하면 길고 가는 행렬과 작고 넓은 행렬로 분해됨
- 분해된 행렬은 잠재 요소를 특성으로 가짐
- W는 원본 행에 대해 이 잠재 요소의 값이 얼마나 되는지에 대응
- H는 이 잠재 요소가 원본 열로 어떻게 구성됐는지를 나타냄

```
from sklearn.decomposition import NMF
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
%matplotlib inline

iris = load_iris()
iris_ftrs = iris.data
nmf = NMF(n_components = 2)
nmf.fit(iris_ftrs)
iris_nmf = nmf.transform(iris_ftrs)

plt.scatter(x=iris_nmf[:, 0], y=iris_nmf[:, 1], c=iris.target)
plt.xlabel('NMF Component 1')
plt.ylabel('NMF Component 2')

/usr/local/lib/python3.11/dist-packages/sklearn/decomposition/_nmf.py:1742: ConvergenceWarning:
  warnings.warn(
Text(0, 0.5, 'NMF Component 2')
```

- NMF도 SVD와 유사하게 이미지 압축 통한 패턴 인식, 텍스트 토픽 모델링 기법, 문서 유사도 등에 잘 사용
- 영화 추천 같은 Recommendation 영역에 활발히 적용
- 사용자의 상품 평가 데이터 세트인 사용자-평가-순위 데이터 세트를 행렬 분해 기법을 통해 분해, 사용자가 평가하지 않은 상품에 대한 잠재적 요소 추출

- 이를 통해 평가 순위 예측 및 추천